5. Arrays, Slices, and Maps (Memory Intuition)

05. Arrays, Slices, and Maps (Memory Intuition)

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1–2 days
Slug: `05-arrays-slices-maps`

Learning objectives

- Distinguish arrays (fixed) from slices (dynamic views)
- Explain slice header: pointer, length, capacity
- Grow slices with append and understand reallocation
- Use maps safely (including comma-ok idiom)

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Slices and maps are how Go holds collections. The slice header (ptr, len, cap) prevents aliasing bugs and needless allocations when you understand it.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Arrays have fixed length in the type; slices are descriptors over arrays.
- `append` may reallocate; sub-slices often share a backing array.
- Maps need `make` or a literal; reading a missing key yields the zero value.
- Comma-ok: `v, ok := m[k]` distinguishes missing from zero.
- `make([]T, n, c)` sets length n and capacity c — pre-size when you know n.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-arrays, && cd /tmp/gw-arrays,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

func main() {
    s := []int{1, 2, 3}
    s = append(s, 4)
    m := map[string]int{"a": 1}
    v, ok := m["b"]
    fmt.Println(s, len(s), cap(s), v, ok)
    t := s[1:3]
    t[0] = 99
    fmt.Println(s, t)
}
```

Run it:

```
go run .
```

Expected output:

```
[1 2 3 4] 4 6 0 false
[1 99 3 4] [99 3]
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Slice s[1:3], mutate, print original — observe sharing.
2. Append until capacity grows; print len/cap before and after.
3. Build a frequency map with m[w]++.
4. Use make([]int, 0, 100) and append in a loop; compare allocations mentally.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Arrays

```go
var a [3]int // [0,0,0]; length is part of the type
```

Rarely used directly in APIs; slices dominate.

Slices — the workhorse

A slice is a **descriptor**: `(pointer to array, length, capacity)`.

```go
s := []int{1, 2, 3}
s = append(s, 4)
t := s[1:3] // shares backing array — mutations can alias
```

When `append` exceeds capacity, Go allocates a new array and copies. That is why sometimes modifying a sub-slice surprises you.

Maps

```
m := map[string]int{"a": 1}
v, ok := m["b"] // ok == false, v == 0
delete(m, "a")
```

Maps are reference types; passing a map to a function lets the callee mutate entries. Never assume iteration order is stable across runs (since Go 1 it is randomized).

Failure modes to watch while coding

- Thinking slices always copy (they often share memory)
- Reading map missing key without comma-ok and treating 0 as real data
- Appending in a loop without pre-sizing when size is known (`make([]T, 0, n)`)

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python lists own storage; Go sub-slices often share memory.
- dict.get vs comma-ok — use ok when zero is valid data.
- Map iteration order is randomized; never rely on it for tests.
- No list comprehensions — use loops or small helper funcs (clear > clever).

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

When returning a slice from a function, consider whether callers can mutate shared backing storage. Sometimes `append([]T(nil), s...)` copies.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Arrays, Slices, and Maps (Memory Intuition)” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked

example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Thinking slices always copy (they often share memory)
- Reading map missing key without comma-ok and treating 0 as real data
- Appending in a loop without pre-sizing when size is known (`make([]T, 0, n)`)

Exercises

Work in `exercises/05-arrays-slices-maps/`. Read that folder's `README.md` before you code.

What you will build

Implement Sum, Freq (word counts), and Unique (first-seen order).

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/05-arrays-slices-maps/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What are the three fields of a slice header?
2. Why can `append` invalidate assumptions about shared backing arrays?
3. How do you distinguish missing key from zero value?

Further reading

- https://go.dev/blog/slices-intro
- https://go.dev/blog/maps